

# NT219 — Cryptography

## Week 4: Modern Symmetric Ciphers

PhD. Ngoc-Tu Nguyen  
tunn@uit.edu.vn

September 29, 2025

# Outline

- 1 Foundations
- 2 Classical Ciphers & Cryptanalysis
- 3 Stream Ciphers
- 4 Block Ciphers
- 5 Feistel Ciphers
- 6 Data Encryption Standard (DES)
- 7 Block Cipher Design Principles
- 8 AES (Preview)

# What is Cryptography?

- **Cryptology** = Cryptography + Cryptanalysis
- **Security Properties:**
  - Confidentiality
  - Integrity
  - Authentication
  - Non-repudiation (Accountability)
  - Availability
  - Privacy

# Cryptographic Primitives (Grouped)

**Encryption (Confidentiality)**

Symmetric (AES, ChaCha20)

Asymmetric (RSA, ECC, CRYSTALS-KYBER)

**Integrity & Authentication**

Hash functions (SHA-2, SHA-3)

Message Authentication Codes (HMAC, CMAC)

**Authentication & Non-repudiation**

Digital Signatures (RSA, ECDSA, Dilithium)

Digital Certificates (PKI)

**Key Establishment**

Diffie–Hellman (DH), Elliptic Curve DH (ECDH), Key Encapsulation Mechanisms (KEMs)

# Classical cipher algorithms

## Substitution techniques

- **Monoalphabetic cipher:** uses a single substitution alphabet (e.g., Caesar, simple substitution).
- **Polyalphabetic cipher:** uses multiple *substitution alphabets* to obscure frequency patterns (e.g., Vigenère).
- **Polygraphic cipher:** substitutes blocks of letters instead of single characters (e.g., Hill cipher, Playfair).

## Transposition techniques

- Characters remain the same but their positions are rearranged.
- Examples: Rail fence, Columnar transposition.

# Cryptanalysis — Statistical techniques

- **Data & notation.** Total count  $N = \sum_i f_i$ ; empirical frequencies  $q_i = f_i/N$ ; language frequencies  $p_i$  (e.g., English).

- **Index of Coincidence (IoC).**

$$\text{IoC} = \frac{\sum_i f_i(f_i - 1)}{N(N - 1)}, \quad \text{IoC}_{\text{EN}} \approx 0.066, \quad \text{IoC}_{\text{rand}} = \frac{1}{26} \approx 0.0385.$$

*Heuristic:*  $\text{IoC} \approx 0.06$  suggests monoalphabetic;  $\approx 0.04$  suggests long-key polyalphabetic.

- **Friedman key-length estimate (Vigenère).**

$$\hat{k} \approx \frac{0.0265 N}{(0.065 - \text{IoC}) + N(\text{IoC} - 0.0385)}.$$

- **Caesar/shift recovery (also helpful for simple monoalphabetic).**

$$\chi^2(s) = \sum_i \frac{(N q_i^{(s)} - N p_i)^2}{N p_i} = \sum_i \frac{(q_i^{(s)} - p_i)^2}{p_i}, \quad s^* = \arg \min_s \chi^2(s).$$

*Alternative score:*  $c(s) = \sum_i q_i^{(s)} p_i$ , take  $\arg \max_s c(s)$ .

- **Notes.** Results stabilize for  $N \gtrsim 200$ ; use digram/trigram tests to break ties or confirm key hypotheses

# Cryptanalysis — Polyalphabetic ciphers

- Example: Vigenère cipher.
- Kasiski examination: find repeated substrings, compute GCD of gaps  $\rightarrow$  key length.
- Friedman test: IoC-based estimate of key length.
- Once key length known: split into columns and apply frequency analysis.
- Known-plaintext (crib-dragging) speeds up recovery.

# Cryptanalysis — Polygraphic ciphers

- Hill cipher:
  - Ciphertext blocks:  $C = KP \pmod{26}$ .
  - With  $n$  plaintext/ciphertext pairs, solve  $K = CP^{-1} \pmod{26}$ .
  - Requires  $\gcd((\det(P) \pmod{26}), 26) = 1$ .
- Playfair cipher:
  - Digraph substitution.
  - Attacked via digraph frequency and filler patterns.



# Cryptanalysis — Transposition ciphers

- Columnar and Rail Fence ciphers.
- Ciphertext is permutation of plaintext characters.
- Attack: guess key length, reconstruct columns/rails.
- Use n-gram scoring to recognise correct plaintext.

# Cryptanalysis — Heuristic approaches

- Hill-climbing, simulated annealing, genetic algorithms.
- Useful for monoalphabetic substitution and mixed ciphers.
- Fitness function: n-gram statistics, dictionary match, crib alignment.

# Attack workflows

- 1 Compute statistics (IoC, frequency, n-grams) to classify cipher family.
- 2 Monoalphabetic  $\rightarrow$  substitution solver.
- 3 Polyalphabetic  $\rightarrow$  estimate key length, split, apply frequency.
- 4 Hill cipher  $\rightarrow$  collect  $n$  pairs, solve linear system mod 26.
- 5 Transposition  $\rightarrow$  enumerate permutations, use scoring.

# Complexity considerations

- Monoalphabetic substitution:  $26!$  keys (infeasible brute force) but solved by statistics.
- Vigenère: reduces to  $k$  monoalphabetic problems; feasible for small  $k$ .
- Hill cipher: polynomial-time linear algebra, but weak for small  $n$ .
- Transposition: factorial search space, pruned by language scoring.

# Countermeasures and lessons

- Classical ciphers insecure against modern analysis.
- Use strong, modern primitives (AES-GCM, ChaCha20-Poly1305).
- Random, long keys and no reuse.
- Authenticated encryption to prevent tampering.
- Classical methods useful for education and CTFs.

# Practical tips for students

- Start with statistics (IoC, frequency) to classify cipher.
- Use scripts with n-gram scoring and heuristic search.
- Leverage cribs (probable plaintexts) when available.
- For Hill cipher: ensure modular inverses exist before solving.

# Stream cipher model (1/2)

- Bit/byte-wise encryption using a keystream.
- Ciphertext  $C = P \oplus \text{KS}$  where KS is keystream.
- Keystream derived from secret key and usually an IV/nonce.<sup>1</sup>

---

<sup>1</sup>Terminology: secret seed / key, IV or nonce, session key (per session in networks).

# ChaCha20: Initialization

We define the state as a  $4 \times 4$  matrix of 32-bit words:

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix}$$

## Initialization (round 0):

$$a_{00..03} = (0x61707865, 0x3320646e, 0x79622d32, 0x6b206574),$$

$$a_{10..13} = (k_0, k_1, k_2, k_3),$$

$$a_{20..23} = (k_4, k_5, k_6, k_7),$$

$$a_{30..33} = (\text{counter}, n_0, n_1, n_2).$$



## ChaCha20 rounds (1/3): Column round

**Column round:** apply the quarter-round to the 4 columns:

$$\begin{aligned} & (a_{00}, a_{10}, a_{20}, a_{30}), \quad (a_{01}, a_{11}, a_{21}, a_{31}), \\ & (a_{02}, a_{12}, a_{22}, a_{32}), \quad (a_{03}, a_{13}, a_{23}, a_{33}) \end{aligned}$$

## ChaCha20 rounds (2/3): Diagonal round

**Diagonal round:** apply the quarter-round to diagonals:

$$\begin{aligned} &(a_{00}, a_{11}, a_{22}, a_{33}), \quad (a_{01}, a_{12}, a_{23}, a_{30}), \\ &(a_{02}, a_{13}, a_{20}, a_{31}), \quad (a_{03}, a_{10}, a_{21}, a_{32}) \end{aligned}$$

**Quarter-round function on  $(a, b, c, d)$ :**

$$\begin{aligned} a &= a + b \pmod{2^{32}}, & d &= (d \oplus a) \lll 16, \\ c &= c + d \pmod{2^{32}}, & b &= (b \oplus c) \lll 12, \\ a &= a + b \pmod{2^{32}}, & d &= (d \oplus a) \lll 8, \\ c &= c + d \pmod{2^{32}}, & b &= (b \oplus c) \lll 7 \end{aligned}$$

**Total:** 20 rounds = 10 double-rounds (Column + Diagonal).

# Stream cipher model (3/3)

- Security pitfalls:

- Keystream reuse (two-time pad)<sup>2</sup>  $\Rightarrow C_1 \oplus C_2 = P_1 \oplus P_2$ .
- Poor PRG design<sup>3</sup>  $\Rightarrow$  state recovery under statistical biases.
- Weak or predictable IV/nonce selection<sup>4</sup>  $\Rightarrow$  replay or collision attacks.

- Cryptanalysis of stream ciphers:

- State recovery attacks**<sup>5</sup>: reconstruct internal state from keystream outputs.
- Distinguishing attacks**<sup>6</sup>: detect non-randomness in keystream vs true randomness.
- Correlation attacks**<sup>7</sup>: exploit linear relation between keystream bits and secret key bits.
- Algebraic attacks**<sup>8</sup>: model keystream generation equations and solve for key/state.
- Side-channel attacks**<sup>9</sup>: recover keys from timing, power, or cache leakage.

<sup>2</sup>Sử dụng lại chuỗi khóa, hay "two-time pad", có thể dẫn đến lộ thông tin khi XOR hai ciphertext.

<sup>3</sup>Thiết kế bộ sinh số giả ngẫu nhiên (PRG) kém có thể khiến trạng thái nội bộ bị khôi phục dựa trên các thiên lệch thống kê.

<sup>4</sup>Chọn IV hoặc nonce yếu hoặc dự đoán được có thể dẫn đến tấn công tái phát hoặc va chạm.

<sup>5</sup>Tấn công phục hồi trạng thái: xây dựng lại trạng thái nội bộ từ các output của keystream.

<sup>6</sup>Tấn công phân biệt: phát hiện tính không ngẫu nhiên trong keystream so với chuỗi ngẫu nhiên thật.

<sup>7</sup>Tấn công tương quan: khai thác mối quan hệ tuyến tính giữa các bit keystream và bit khóa bí mật.

<sup>8</sup>Tấn công đại số: mô hình hóa các phương trình sinh keystream và giải để tìm khóa hoặc trạng thái.

<sup>9</sup>Tấn công kênh bên: khôi phục khóa dựa trên thông tin rò rỉ từ thời gian, điện năng hoặc cache.

# Block cipher basics

- Map a *block* of plaintext to a *block* of ciphertext (same length).
- Typical block sizes: 64 or 128 bits.
- Shared symmetric key; widely used in network applications.

# Stream vs. Block ciphers

- **Stream**<sup>10</sup>: bit/byte granularity, low latency, careful nonce handling.
- **Block**<sup>11</sup>: block granularity, modes of operation (ECB, CBC, CTR, GCM, ...).
- Choice depends on use case (throughput, latency, AEAD needs).

---

<sup>10</sup>Granularity: mức độ chi tiết xử lý dữ liệu. Ở cipher dạng stream, dữ liệu được xử lý theo từng bit hoặc byte.

<sup>11</sup>Ở cipher dạng block, dữ liệu được xử lý theo từng khối cố định (block).

# Substitution tables

| Type                        | Input granularity          | Description / Key space                    |
|-----------------------------|----------------------------|--------------------------------------------|
| Monoalphabetic substitution | bit / byte (single symbol) | One-to-one mapping of each symbol          |
| Block substitution (S-box)  | $n$ -bit block             | Maps $n$ -bit block to $n$ -bit block; key |

**Table:** Comparison of substitution types and their key space

## Question

How many possible substitutions exist for an  $n$ -bit block?

Answer:

## Feistel structure: intuition

- Alternates **substitutions**<sup>12</sup> and **permutations**<sup>13</sup>.
- Implements Shannon's **confusion**<sup>14</sup> & **diffusion**<sup>15</sup>.
- Popular for many block ciphers, e.g., DES<sup>16</sup>, Blowfish<sup>17</sup>, CAST-128<sup>18</sup>, IDEA (modified Feistel)<sup>19</sup>, Twofish<sup>20</sup>.

<sup>12</sup>Substitution: thay thế dữ liệu, ví dụ ánh xạ ký tự trong Caesar cipher hoặc khối trong S-box.

<sup>13</sup>Permutation: tráo đổi vị trí dữ liệu, ví dụ tráo thứ tự bit hoặc byte trong cipher block.

<sup>14</sup>Confusion: làm phức tạp mối quan hệ giữa khóa và ciphertext, giảm khả năng suy đoán khóa.

<sup>15</sup>Diffusion: phân tán thông tin plaintext khắp ciphertext, giảm tính lộ dữ liệu.

<sup>16</sup>DES: Data Encryption Standard, cipher khối cổ điển sử dụng Feistel với 16 vòng.

<sup>17</sup>Blowfish: cipher khối sử dụng Feistel với 16 vòng, block 64-bit, khóa biến đổi từ 32–448 bit.

<sup>18</sup>CAST-128: cipher khối Feistel 64-bit, 12–16 vòng, dùng trong PGP và TLS.

<sup>19</sup>IDEA: sử dụng cấu trúc gần giống Feistel, block 64-bit, 8.5 vòng; nổi bật trong PGP.

<sup>20</sup>Twofish: cipher khối 128-bit, Feistel-style, khóa 128/192/256-bit, an toàn, từng là ứng viên AES.

# Confusion and diffusion (Shannon)

## Diffusion<sup>a</sup>

<sup>a</sup>Diffusion: phân tán thông tin plaintext khắp ciphertext, làm giảm khả năng rút trích thông tin thống kê.

**Example:** Plaintext: 1010 1100 0110 1001 After diffusion (simplified): 1101 0011 1010 0110

- Một bit thay đổi trong plaintext ảnh hưởng nhiều bit trong ciphertext.
- Thống kê ký tự/bit ban đầu không còn trực tiếp nhận ra ((as in **DES or AES diffusion algorithm**)).

## Confusion<sup>a</sup>

<sup>a</sup>Confusion: làm phức tạp mối quan hệ giữa key và ciphertext, giảm khả năng suy đoán khóa.

**Example:** Ciphertext =  $F(\text{plaintext}, K)$

- Thay đổi 1 bit của khóa  $K \rightarrow$  nhiều bit ciphertext thay đổi không dự đoán được.
- Giúp bảo vệ khóa khỏi các tấn công phân tích thống kê.



# Feistel encryption/decryption (16 rounds)

## Setup:

- Block size:  $2\ell$  bits, split  $M = L_0 \| R_0$  with  $|L_0| = |R_0| = \ell$ .
- Number of rounds:  $n = 16$  (example for DES).
- Round keys:  $K_1, K_2, \dots, K_n$ .

## Encryption (round $i = 1, \dots, n$ ):

$$L_i = R_{i-1} \quad \text{(left becomes previous right)}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i) \quad \text{(right = previous left XOR round function)}$$

## Ciphertext:

$$C = L_{n+1} \| R_{n+1} \quad \text{with } L_{n+1} = R_n, R_{n+1} = L_n$$

## Decryption:

- Same structure as encryption.
- Use subkeys in reverse order:  $K_n, K_{n-1}, \dots, K_1$ .

**Tip for visualization:** Draw L/R boxes and arrows per round to see the swap and F-function effect.

## Correctness proof (sketch)

By induction, show for decryption state  $L'_i = R_{n-i}$  and  $R'_i = L_{n-i}$  for  $i = 0, \dots, n$ ; thus  $L'_{n+1} \| R'_{n+1} = L_0 \| R_0 = M$ .

# Feistel design features

- **Block size:** larger generally  $\rightarrow$  better security, slower speed.
- **Key size:** larger  $\rightarrow$  better brute-force resistance.
- **#Rounds:** security grows with rounds (single round is inadequate).
- **Round function  $F$ :** nonlinearity/complexity improves resistance.
- **Subkey generation:** make recovering subkeys/main key hard.
- **Implementation:** fast software paths, clear specification to aid analysis.

# DES overview

- FIPS 46 (1977); dominant until AES (2001).
- 64-bit block; 56-bit key (64 bits with parity).
- 16-round Feistel; same key for encryption/decryption.

# DES encryption outline

## Setup:

- Input plaintext  $M$  (64-bit)
- Initial permutation:  $IP(M) = L_0 || R_0$  with  $|L_0| = |R_0| = 32$
- Subkeys:  $K_1, K_2, \dots, K_{16}$

## Encryption rounds ( $i = 1, \dots, 16$ ):

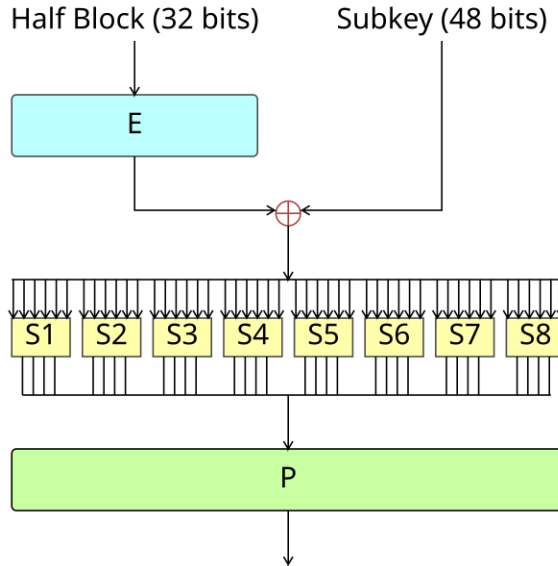
$$\begin{aligned}
 L_i &= R_{i-1} && \text{(left half becomes previous right)} \\
 R_i &= L_{i-1} \oplus F(R_{i-1}, K_i) && \text{(right = previous left XOR round function)}
 \end{aligned}$$

## Output:

- Preoutput:  $R_{16} || L_{16}$  (note the swap)
- Ciphertext:  $C = IP^{-1}(R_{16} || L_{16})$

**Tip:** Draw L/R boxes and arrows per round to visualize Feistel structure.

# DES round function F

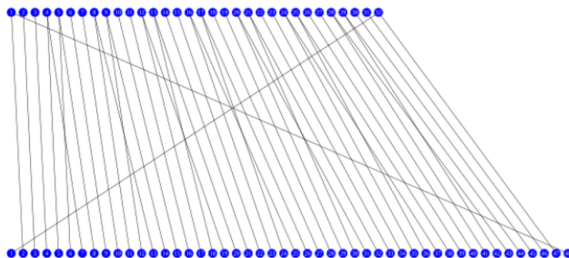


# DES round function $F$

$$F(R_{i-1}, K_i) = P(S(EP(R_{i-1}) \oplus K_i)).$$

- Expansion EP :  $32 \rightarrow 48$  bits; XOR with  $K_i$ .
- Eight S-boxes:  $8 \times (6\text{-bit} \rightarrow 4\text{-bit})$  mappings  $\Rightarrow 32$  bits.
- Final permutation  $P$  of 32 bits.

| E  |    |    |    |    |    |
|----|----|----|----|----|----|
| 32 | 1  | 2  | 3  | 4  | 5  |
| 4  | 5  | 6  | 7  | 8  | 9  |
| 8  | 9  | 10 | 11 | 12 | 13 |
| 12 | 13 | 14 | 15 | 16 | 17 |
| 16 | 17 | 18 | 19 | 20 | 21 |
| 20 | 21 | 22 | 23 | 24 | 25 |
| 24 | 25 | 26 | 27 | 28 | 29 |
| 28 | 29 | 30 | 31 | 32 | 1  |





# DES S-box Lookup (Example)

**Input:** 6 bits  $b_1 b_2 b_3 b_4 b_5 b_6$

**Step 1: Determine row and column**

- Row:  $b_1 b_6$  (2 bits, 0–3)
- Column:  $b_2 b_3 b_4 b_5$  (4 bits, 0–15)

**Step 2: Lookup S-box table ( $S_i$  for block  $i=1\dots 8$ )**

- Use the row and column to find the 4-bit output.

**Example ( $S_5$ ):**

- Input:  $b = 101011$
- Row:  $b_1 b_6 = 11_2 = 3$
- Column:  $b_2 b_3 b_4 b_5 = 0101_2 = 5$
- Output from  $S_5$ :  $1001_2$  (example)

21

<sup>21</sup>DES S-boxes: 8 tables, each maps 6-bit input  $\rightarrow$  4-bit output to provide nonlinearity.

# DES Lookup table S-box

**Input:** 011011  $\rightarrow$  0 1101 1

**row:** 01; **column:** 1101

**Output:**  $S_{01,1101} = 1001$

| S <sub>5</sub> |    | Middle 4 bits of input |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|----------------|----|------------------------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
|                |    | 0000                   | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |
| Outer bits     | 00 | 0010                   | 1100 | 0100 | 0001 | 0111 | 1010 | 1011 | 0110 | 1000 | 0101 | 0011 | 1111 | 1101 | 0000 | 1110 | 1001 |
|                | 01 | 1110                   | 1011 | 0010 | 1100 | 0100 | 0111 | 1101 | 0001 | 0101 | 0000 | 1111 | 1010 | 0011 | 1001 | 1000 | 0110 |
|                | 10 | 0100                   | 0010 | 0001 | 1011 | 1010 | 1101 | 0111 | 1000 | 1111 | 1001 | 1100 | 0101 | 0110 | 0011 | 0000 | 1110 |
|                | 11 | 1011                   | 1000 | 1100 | 0111 | 0001 | 1110 | 0010 | 1101 | 0110 | 1111 | 0000 | 1001 | 1010 | 0100 | 0101 | 0011 |

## Key schedule (summary)

- Start from  $K = k_1 \dots k_{64}$ ; remove parity bits  $\Rightarrow$  56 bits.
- Initial key permutation to  $U_0 \parallel V_0$  (28 bits each).
- Round-dependent circular left shift:  $U_i = \text{ROTL}_{z(i)}(U_{i-1})$ ,  $V_i = \text{ROTL}_{z(i)}(V_{i-1})$ .
- Compression permutation to  $K_i$  (48 bits). **Left shifts per round  $z(i)$ :**

|           |   |    |    |    |    |    |    |    |
|-----------|---|----|----|----|----|----|----|----|
| Round $i$ | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| $z(i)$    | 1 | 1  | 2  | 2  | 2  | 2  | 2  | 2  |
| Round $i$ | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| $z(i)$    | 1 | 2  | 2  | 2  | 2  | 2  | 2  | 1  |

# Is DES still secure?

- Key space:  $2^{56} \approx 7.2 \times 10^{16}$  keys.
- Brute-force milestones:
  - 1997: Internet collaborative effort (months).
  - 1998: EFF custom hardware (days).
  - 1999: Combined approach ( $\sim 22$  hours).
- Conclusion: DES alone is obsolete; use 3DES (legacy) or AES.

# DES cryptanalysis: classical results

- **Differential cryptanalysis** (Biham–Shamir, 1993): first published attack on full 16-round DES better than brute force; uses about  $2^{47}$  chosen plaintexts to collect  $\approx 2^{36}$  useful pairs; time  $\approx 2^{37}$  encryptions.<sup>22</sup>
- **Linear cryptanalysis** (Matsui, 1993/94): known-plaintext attack; needs about  $2^{43}$  known plaintexts to recover a 16-round DES key; first experimental key recovery reported.<sup>23</sup>
- **Davies–Murphy attack**: exploits biased joint distributions of adjacent S-box outputs; improved to about  $2^{50}$  known-plaintexts (Biham–Biryukov, 1997) and to about  $2^{45}$  chosen-plaintexts (Kunz-Jacques & Muller, 2005).<sup>24</sup>

<sup>22</sup>E. Biham and A. Shamir, *Differential Cryptanalysis of the Data Encryption Standard*, Springer, 1993; authors' LaTeX version (pp. 1–2).

<sup>23</sup>M. Matsui, “Linear Cryptanalysis Method for DES Cipher” (abstract).

<sup>24</sup>E. Biham and A. Biryukov, *Journal of Cryptology* (1997); S. Kunz-Jacques and F. Muller, “New Improvements of Davies–Murphy Cryptanalysis,” ASIACRYPT 2005.

# DES properties & practical implications

- **Weak / semi-weak keys and complementation property:** 4 weak keys and 6 semi-weak key pairs;  $E_K(P) = C \Rightarrow E_{\bar{K}}(\bar{P}) = \bar{C}$ , which can halve exhaustive search under chosen-plaintext.<sup>25</sup>
- **Double DES is broken by meet-in-the-middle:** effective complexity  $\approx 2^{57}$  time with  $\approx 2^{56}$  space; motivates 3DES instead of 2DES.<sup>26</sup>
- **Practical brute force:** EFF *Deep Crack* (1998) broke DES in  $\sim 56$  hours; EFF + distributed.net (1999) in  $\sim 22$  hours.<sup>27</sup>
- **Current status:** DES is obsolete; TDEA (3DES) withdrawn for new use by NIST.<sup>28</sup>

<sup>25</sup> *Data Encryption Standard* (Security and cryptanalysis); D. W. Davies, "Some Regular Properties of the DES," CRYPTO '82.

<sup>26</sup> "Meet-in-the-middle attack" (Double DES) overview.

<sup>27</sup> EFF press releases on DES Challenge II/III; "EFF DES cracker" historical summary.

<sup>28</sup> NIST CSRC announcement, "NIST to Withdraw SP 800-67 Rev. 2" (effective Jan. 1, 2024); SP 800-131A transition guidance.

# Principles (I)

- #Rounds must exceed effort of brute-force search under best-known cryptanalysis.
- Round function should satisfy:
  - **SAC** (Strict Avalanche Criterion): each output bit flips with prob.  $\frac{1}{2}$  when any single input bit is inverted.
  - **BIC** (Bit Independence Criterion): output bits change independently under single input-bit inversions.

## Principles (II)

- Key schedule should maximize difficulty of deducing subkeys and recovering the master key.
- Algorithm should have good avalanche effect overall.
- Favor designs amenable to fast software while remaining analyzable.



# Advanced Encryption Standard (teaser)

- 128-bit block cipher with key sizes 128/192/256.
- Substitution–permutation network (not Feistel).
- We will cover AES structure, MixColumns, SubBytes, ShiftRows, and key schedule in Part 2.

# References & Reading

- *Applied Cryptography, Cryptography and Network Security*, and standard lecture notes for Chapter 4–6 (block/stream ciphers) and Chapter 5 (DES/AES).
- Feistel, H. (1973). *Scientific American* 228(5), 15–23.
- NIST FIPS 46 (DES) and its supplements.